Sprint 3 Team Retrospective

Project: B2B System Health Dashboard | ELEC5619, Lab 02, Group 02

Review date: 19 October 2026

Team: Dakota Wu, Zhouguang Li, Yixuan Huang, Zeheng Li and Wentao Chen

Sprint goal and outcome

Sprint 3 focused on completing the operational workflow across the React frontend, Spring Boot API and MySQL database. The increment connects system registration and editing, latest measurements, configured health thresholds, dashboard averages, active-alert counts and incident handling. Operations users can submit manual observations; Management users can inspect monitoring data without operational write permissions. Users can also maintain their own email address and verify it through the configured email service.

PRs #15, #16, #17 and #18 are integrated into develop-3. PR #19 extends database validation, transaction handling, system/history pagination, optional metric archiving and profile verification. Its follow-up combines the merged frontend with these additions and corrects the review findings. Updating a PR remains separate from merging it or accepting the final project.

What went well

- **The increment became demonstrable from the interface.** Registering a system, entering a reading, inspecting the resulting health/KPIs and processing an alert now form a connected workflow. Earlier backend capabilities are visible to the user.
- **Reviews improved domain correctness.** The integrated changes preserve unique system identities, expose effective thresholds and distinguish stored health from FRESH, STALE and NO_DATA. Acknowledged alerts are not counted as ACTIVE or confused with resolved incidents.
- **Security remained part of integration.** Server-side role checks and CSRF protection accompany the UI controls. Personal-profile access is scoped to the authenticated user, while operational writes remain restricted.
- **Verification became more reproducible.** Java 21 checks, versioned migrations, a Docker verification entrypoint and focused regression tests provide stronger evidence than screenshots or a successful build alone.

Challenges and lessons

Observation	Impact and lesson
PR #19 included frontend changes that overlapped PR #18; integrating the merged version produced eight conflicted files.	Choosing one complete file version would discard working features. Integrate smaller changes sooner, agree component ownership and review the combined behaviour after conflict resolution.
A slow initial history request could overwrite a newer time-range result.	Individual successful requests did not guarantee a correct screen. Initial loading, filtering and pagination need a shared cancellation boundary, with a delayed-response regression test.
A history test used timestamps with finer precision than the database stores.	The same test failed on another development environment. Test fixtures and cursor keys must respect database precision; realistic request boundaries matter when evaluating persistence behaviour.
Some additional frontend validation remained local, and historical evidence described a different source snapshot.	A local pass cannot be reproduced by reviewers unless the tests are committed. Keep dated evidence tied to its tested revision and publish a separate result after integration changes.

Follow-through from Sprint 2

The earlier call for complete vertical slices is reflected in system management, manual metric entry and the visible alert workflow. The Java-version risk is addressed in this verification run by explicitly using Java 21. However, smaller PRs and evidence updates still need stronger follow-through: the large integration conflict and locally retained tests show that these practices are not yet consistent. Jira completion, meeting attendance and rehearsal outcomes require their own records and are not inferred from commits.

Actions for the next integration cycle

These actions assign responsibility by role for the team to adopt; they do not record an unverified meeting decision.

Action	Responsible role	When and completion measure
Integrate one user-visible change at a time; synchronise with develop-3 before requesting review.	Feature author and reviewer	Every next PR: related UI/API changes are identified, conflicts resolved and combined behaviour checked.
Commit regression tests with each fix, including delayed responses and mutation failures.	Feature author	Before merge: a fresh checkout runs the same tests used as evidence; no essential test is local-only.
Re-run the Docker MySQL/Mailpit suite against the updated PR #19 revision.	Backend/database owner and reviewer	Before accepting #19: record the tested SHA, command, environment and all results; do not reuse the earlier snapshot's pass.
Rehearse the Operations and Management workflows from a clean setup.	Integration coordinator and all members	Before the final demo: record two timed rehearsals, include restart persistence, error/retry paths and role restrictions.
Finish accessibility and user-task evaluation, and align Jira/contribution links with delivered work.	QA owner and each issue owner	Before submission: retain keyboard/contrast checks, actual user-task results and traceable contribution evidence.

Verification and remaining evidence

The 19 October follow-up passes **158 backend tests**, the **H2 migration check**, **64 frontend tests** and the **frontend production build**. The added frontend tests cover the history-request race, system creation/editing, metric-to-dashboard/alert refresh, Management controls, filtering beyond the visible systems page and logout failure/retry. Current integration verification is recorded separately in PR #19 follow-up verification.

The original PR #19 author recorded Docker MySQL/Mailpit and browser results for the earlier source manifest. Those remain historical evidence. This follow-up's standalone MySQL checks establish the listed database/API behaviours, but do not replace the Docker suite or prove external email delivery, production performance, broad accessibility or user acceptance.

Overall reflection

Sprint 3's main improvement is the move from separate implemented components to an integrated operational workflow. The main process lesson is to integrate and verify continuously: a feature is ready for review when its interface, API, persisted behaviour, failure paths and reproducible evidence agree. The final demonstration should make that agreement visible without presenting outstanding acceptance work as complete.